

Q.1: Implement the stack using queue. Use one or two test cases to perform the push() and pop() operations using the queue and show the output with respect to the utilized queues.

```
#include <iostream>
#include <queue>
using namespace std;

class StackUsingQueue {
    queue<int> q1, q2;

public:
    void push(int x) {
        cout << "Pushing " << x << endl;
        q2.push(x);
        while (!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
        swap(q1, q2);
    }

    void pop() {
        if (q1.empty()) {
            cout << "Stack is Empty\n";
            return;
        }
        cout << "Popped " << q1.front() << endl;
        q1.pop();
    }

    void display() {
        cout << "Current Stack (top to bottom): ";
        queue<int> temp = q1;
        while (!temp.empty()) {
            cout << temp.front() << " ";
            temp.pop();
        }
        cout << endl;
    }
};

int main() {
    StackUsingQueue s;
```

```

    s.push(10);
    s.push(20);
    s.push(30);
    s.display();
    s.pop();
    s.display();
    cout << endl;
    return 0;
}

```

```

1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  class StackUsingQueue {
6      queue<int> q1, q2;
7
8  public:
9      void push(int x) {
10         cout << "Pushing " << x << endl;
11         q2.push(x);
12         while (!q1.empty()) {
13             q2.push(q1.front());
14             q1.pop();

```

Pushing 10
Pushing 20
Pushing 30
Current Stack (top to bottom): 30 20 10
Popped 30
Current Stack (top to bottom): 20 10

```

Pushing 10
Pushing 20
Pushing 30
Current Stack (top to bottom): 30 20 10
Popped 30
Current Stack (top to bottom): 20 10

```

Q.2: Implement the queue using stack. Use one or two test cases to perform the enqueue() and dequeue() operations using the stack and show the output with respect to the utilized stacks.

```

#include <iostream>
#include <stack>

class QueueUsingStack {
private:
    std::stack<int> s1;
    std::stack<int> s2;

public:

```

```

void enqueue(int x) {
    std::cout << "Enqueue: " << x << std::endl;
    s1.push(x);
}

int dequeue() {
    if (s1.empty() && s2.empty()) {
        std::cout << "Queue is empty." << std::endl;
        return -1;
    }

    if (s2.empty()) {
        while (!s1.empty()) {
            s2.push(s1.top());
            s1.pop();
        }
    }
    int dequeued_val = s2.top();
    s2.pop();
    std::cout << "Dequeued: " << dequeued_val << std::endl;
    return dequeued_val;
}

void display() {
    std::stack<int> temp_s2 = s2;
    std::stack<int> temp_s1 = s1;
    std::cout << "Current Queue (front to rear): ";

    while (!temp_s2.empty()) {
        std::stack<int> display_temp;
        while (!temp_s2.empty()) {
            display_temp.push(temp_s2.top());
            temp_s2.pop();
        }
        while (!display_temp.empty()) {
            std::cout << display_temp.top() << " ";
            display_temp.pop();
        }
    }

    if (!s1.empty()) {
        std::stack<int> display_s1_temp;
        while (!temp_s1.empty()) {
            display_s1_temp.push(temp_s1.top());
            temp_s1.pop();
        }
        while (!display_s1_temp.empty()) {
            std::cout << display_s1_temp.top() << " ";
            display_s1_temp.pop();
        }
    }
    std::cout << std::endl;
}
};

```

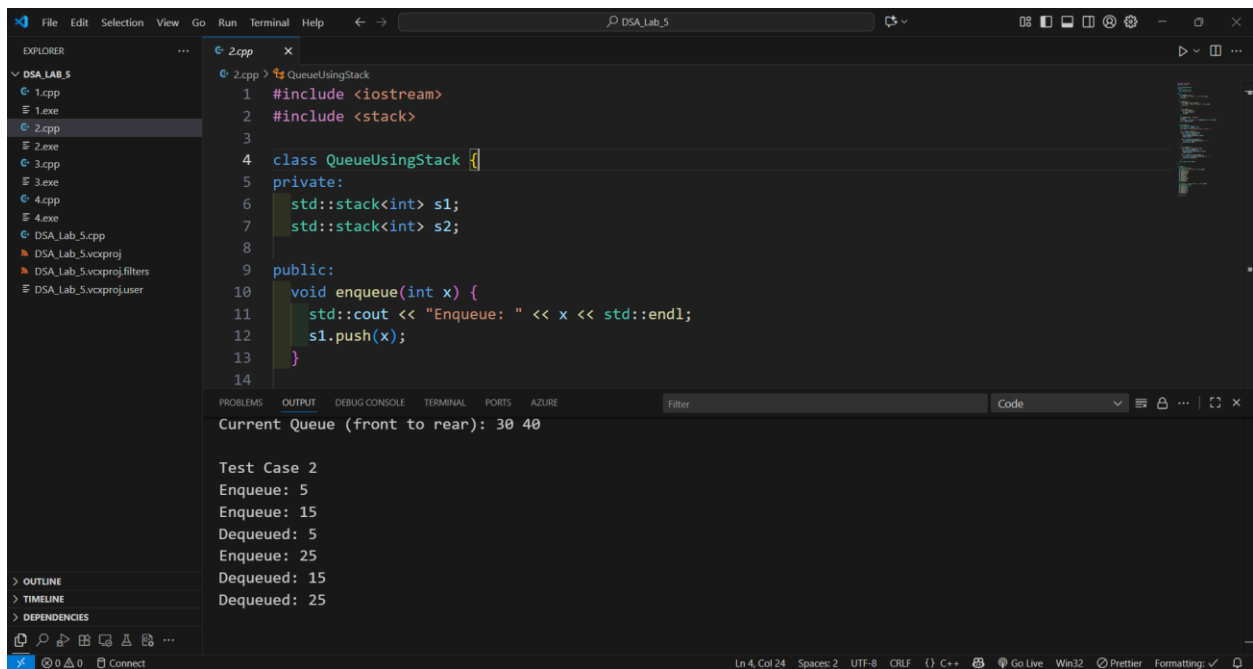
```

int main() {
    QueueUsingStack q1;
    std::cout << "Test Case 1" << std::endl;
    q1.enqueue(10);
    q1.enqueue(20);
    q1.enqueue(30);
    q1.display();
    q1.dequeue();
    q1.display();
    q1.enqueue(40);
    q1.dequeue();
    q1.display();

    std::cout << "\nTest Case 2 " << std::endl;
    QueueUsingStack q2;
    q2.enqueue(5);
    q2.enqueue(15);
    q2.dequeue();
    q2.enqueue(25);
    q2.dequeue();
    q2.dequeue();

    return 0;
}

```



Test Case 1

Enqueue: 10

Enqueue: 20

Enqueue: 30

Current Queue (front to rear): 10 20 30

Dequeued: 10

Current Queue (front to rear): 30 20

Enqueue: 40

Dequeued: 20

Current Queue (front to rear): 30 40

Test Case 2

Enqueue: 5

Enqueue: 15

Dequeued: 5

Enqueue: 25

Dequeued: 15

Dequeued: 25

Q.3: Write a program to evaluate the infix expression using the stack. Show the outputs for two or more test cases. Output should contain the evaluated value along with the intermediate postfix expression. (Use only a single stack at a time)

```
#include <iostream>
#include <stack>
#include <string>
#include <cctype>
#include <vector>
#include <sstream>
#include <cmath>

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}
```

```

std::string infix_to_postfix(const std::string& expression) {
    std::stack<char> s;
    std::string output = "";
    for (char ch : expression) {
        if (isdigit(ch)) {
            output += ch;
        }
        else if (ch == '(') {
            s.push(ch);
        }
        else if (ch == ')') {
            while (!s.empty() && s.top() != '(') {
                output += s.top();
                s.pop();
            }
            if (!s.empty()) s.pop();
        }
        else {
            while (!s.empty() && s.top() != '(' && precedence(s.top()) >=
precedence(ch)) {
                output += s.top();
                s.pop();
            }
            s.push(ch);
        }
    }
    while (!s.empty()) {
        output += s.top();
        s.pop();
    }
    return output;
}

```

```

double evaluate_postfix(const std::string& expr) {
    std::stack<double> s;
    for (char ch : expr) {
        if (isdigit(ch)) {
            s.push(ch - '0');
        }
        else {
            double val2 = s.top(); s.pop();
            double val1 = s.top(); s.pop();
            switch (ch) {
                case '+': s.push(val1 + val2); break;
                case '-': s.push(val1 - val2); break;
                case '*': s.push(val1 * val2); break;
                case '/': s.push(val1 / val2); break;
            }
        }
    }
    return s.top();
}

```

```

int main() {
    std::vector<std::string> test_expressions = { "3+(2*4)", "5*(6-2)/2",
    "9/(3-0)" };

    for (const auto& exp : test_expressions) {
        std::string postfix = infix_to_postfix(exp);
        double value = evaluate_postfix(postfix);

        std::cout << "Infix: " << exp
        << " -> Postfix: " << postfix
        << " -> Evaluated Value: " << value << std::endl;
    }
    return 0;
}

```

The screenshot shows a Visual Studio Code editor with a C++ file named `3.cpp`. The code defines a `precedence` function and a `main` function that tests three expressions. The output window at the bottom shows the results of the program's execution.

```

1 #include <iostream>
2 #include <stack>
3 #include <string>
4 #include <cctype>
5 #include <vector>
6 #include <sstream>
7 #include <cmath>
8
9 int precedence(char op) {
10     if (op == '+' || op == '-') return 1;
11     if (op == '*' || op == '/') return 2;
12     return 0;
13 }
14
15 int main() {
16     std::vector<std::string> test_expressions = { "3+(2*4)", "5*(6-2)/2",
17     "9/(3-0)" };
18
19     for (const auto& exp : test_expressions) {
20         std::string postfix = infix_to_postfix(exp);
21         double value = evaluate_postfix(postfix);
22
23         std::cout << "Infix: " << exp
24         << " -> Postfix: " << postfix
25         << " -> Evaluated Value: " << value << std::endl;
26     }
27     return 0;
28 }

```

Output:

```

Infix: 3+(2*4) -> Postfix: 324*+ -> Evaluated Value: 11
Infix: 5*(6-2)/2 -> Postfix: 562-*2/ -> Evaluated Value: 10
Infix: 9/(3-0) -> Postfix: 930-/ -> Evaluated Value: 3

```

Infix: 3+(2*4) -> Postfix: 324*+ -> Evaluated Value: 11

Infix: 5*(6-2)/2 -> Postfix: 562-*2/ -> Evaluated Value: 10

Infix: 9/(3-0) -> Postfix: 930-/ -> Evaluated Value: 3

Q.4: Given a set of integers, write a program to insert the integers one by one into a stack in such a way that at last the final stack values should have integers in ascending order, starting from the bottom of the stack. (Use only stack operations, not sorting algorithms)

```

#include <iostream>
#include <stack>
#include <vector>
#include <algorithm>
#include <sstream>

void sort_stack(const std::vector<int>& input_list) {
    std::stack<int> main_stack;
    std::stack<int> temp_stack;

    for (int num : input_list) {
        while (!main_stack.empty() && main_stack.top() > num) {
            temp_stack.push(main_stack.top());
            main_stack.pop();
        }

        main_stack.push(num);

        while (!temp_stack.empty()) {
            main_stack.push(temp_stack.top());
            temp_stack.pop();
        }
    }

    std::vector<int> sorted_vec;
    while (!main_stack.empty()) {
        sorted_vec.push_back(main_stack.top());
        main_stack.pop();
    }
    std::reverse(sorted_vec.begin(), sorted_vec.end());

    std::cout << "Final Sorted Stack (bottom -> top): [";
    for (size_t i = 0; i < sorted_vec.size(); ++i) {
        std::cout << sorted_vec[i] << (i == sorted_vec.size() - 1 ? "" : ",
");
    }
    std::cout << "]" << std::endl;
}

void print_input(const std::vector<int>& input) {
    std::cout << "Input set of integers: ";
    for (int val : input) {
        std::cout << val << " ";
    }
    std::cout << std::endl;
}

int main() {

    std::cout << "Test Case 1: Required Example" << std::endl;
    std::vector<int> test1 = { 6, 5, 3, 2, 1, 1, 3, 4, 8, 5, 9, 6, 7 };
    print_input(test1);
    sort_stack(test1);
    std::cout << std::endl;
}

```



```

std::cout << "Test Case 2: Reversed Order" << std::endl;
std::vector<int> test2 = { 10, 8, 6, 4, 2 };
print_input(test2);
sort_stack(test2);
std::cout << std::endl;

std::cout << "Test Case 3: Mixed Negatives and Positives" << std::endl;
std::vector<int> test3 = { 5, -3, 0, 1, -1, 4 };
print_input(test3);
sort_stack(test3);

return 0;
}

```

```

1 #include <iostream>
2 #include <stack>
3 #include <vector>
4 #include <algorithm>
5 #include <sstream>
6
7 void sort_stack(const std::vector<int>& input_list) {
8     std::stack<int> main_stack;
9     std::stack<int> temp_stack;
10
11     for (int num : input_list) {
12         while (!main_stack.empty() && main_stack.top() > num) {
13             temp_stack.push(main_stack.top());
14             main_stack.pop();
15         }
16         main_stack.push(num);
17     }
18     while (!main_stack.empty()) {
19         temp_stack.push(main_stack.top());
20         main_stack.pop();
21     }
22 }
23
24 int main() {
25     // Test Case 1: Required Example
26     std::vector<int> test1 = { 6, 5, 3, 2, 1, 1, 3, 4, 8, 5, 9, 6, 7 };
27     print_input(test1);
28     sort_stack(test1);
29     std::cout << "Final Sorted Stack (bottom -> top): [1, 1, 2, 3, 3, 4, 5, 5, 6, 6, 7, 8, 9]" << std::endl;
30
31     // Test Case 2: Reversed Order
32     std::vector<int> test2 = { 10, 8, 6, 4, 2 };
33     print_input(test2);
34     sort_stack(test2);
35     std::cout << "Final Sorted Stack (bottom -> top): [2, 4, 6, 8, 10]" << std::endl;
36
37     // Test Case 3: Mixed Negatives and Positives
38     std::vector<int> test3 = { 5, -3, 0, 1, -1, 4 };
39     print_input(test3);
40     sort_stack(test3);
41     std::cout << "Final Sorted Stack (bottom -> top): [-3, -1, 0, 1, 4, 5]" << std::endl;
42
43     return 0;
44 }

```

Final Sorted Stack (bottom -> top): [1, 1, 2, 3, 3, 4, 5, 5, 6, 6, 7, 8, 9]

Test Case 2: Reversed Order
Input set of integers: 10 8 6 4 2
Final Sorted Stack (bottom -> top): [2, 4, 6, 8, 10]

Test Case 3: Mixed Negatives and Positives
Input set of integers: 5 -3 0 1 -1 4
Final Sorted Stack (bottom -> top): [-3, -1, 0, 1, 4, 5]

Test Case 1: Required Example

Input set of integers: 6 5 3 2 1 1 3 4 8 5 9 6 7

Final Sorted Stack (bottom -> top): [1, 1, 2, 3, 3, 4, 5, 5, 6, 6, 7, 8, 9]

Test Case 2: Reversed Order

Input set of integers: 10 8 6 4 2

Final Sorted Stack (bottom -> top): [2, 4, 6, 8, 10]

Test Case 3: Mixed Negatives and Positives

Input set of integers: 5 -3 0 1 -1 4

Final Sorted Stack (bottom -> top): [-3, -1, 0, 1, 4, 5]
